

Lab 6: Causal ML for Orange Juice Price Elasticity

Mehreen Ali Gillani

2026-04-13

Table of contents

Overview	1
Problem 1: Testing on Fake Data	2
Problem 2: Checking for Overlap	12
Problem 3: Fitting and Interpreting the Model	14
Problem 4: CATE for Brands and Income	20

Overview

You have been given a dataset consisting of a single file, [oj_large.csv](#). The dataset is a subset of a dataset compiled during a large study by the Chicago Booth School of Business, which collaborated with a local supermarket chain called Dominick’s Finer Foods, to study the impact of prices, advertising, and demographics on the sales of a number of products. The dataset we are working with has over 29,000 observations of the price of orange juice and sales for different brands at different Dominick’s stores. There is a dataset description here: [oj_dictionary.qmd](#).

The goal of this homework assignment is to use Causal Machine Learning to understand how different demographic factors influence something called the ‘price elasticity’ of orange juice. You can read more about price elasticity here: [Wikipedia Price Elasticity of Demand](#). If we define demand as sales, the price elasticity of demand is the relationship between a percentage change in sales and a percentage change in price:

$$\epsilon = \frac{\partial(\text{SALES})}{\partial(\text{PRICE})} \frac{\text{PRICE}}{\text{SALES}}$$

This relationship is most natural when expressed in terms of the ‘log’ transform of both sales and price, as the elasticity becomes the coefficient in a linear regression model relating the two:

$$\log(\text{SALES}) = \epsilon \log(\text{PRICE}) + \text{error terms}$$

The elasticity ϵ can be dependent upon a variety of other factors. It can depend on the price itself (so that we don’t get a straight line relationship between the logs), it can depend on the type of product, the demographics of the shoppers and more. The ‘EconML’ package developed a vignette where they used Causal ML to show that ϵ is a function of income, which is to say that the sales are more sensitive to price in stores where the median income is lower. You can find that vignette [here](#) and I recommend that you read it and use some code from it as a starting point (it covers more than OJ but it is there): [EconML OJ Vignette](#) . To read more about the package and its applications, see [pywhy EconML](#). The [tutorial for this package](#) is helpful as well.

Problem 1: Testing on Fake Data

- (a) It is standard practice in Causal Inference to test models on simulated response data based on the original covariates of the dataset before fitting to the original dataset. Use the same selection of confounders as in the original vignette (The ‘W’ matrix), excluding ‘week’, ‘store’, ‘price’, ‘INCOME’, and ‘logmove’, applying One-Hot encoding/dummies to the ‘brand’ variables to incorporate them into ‘W’. Apply the ‘StandardScaler’ to ‘W’. Then put the variable ‘INCOME’ into a matrix called “X” and standardize it. The matrix ‘X’ contains *modifier* variables whose effect on the elasticity will be studied. Then you will simulate a relationship between your confounders and the price, you can use code like this: ‘T_sim = 0.8 + W[:, support] @ coefs_T + noise’, where ‘support’ is sparse (most entries are 0, the rest are 1) and ‘coefs_T’ is random (the 0.8 is just for scale). Look to the simulation code earlier in the ‘EconML’ vignette for inspiration.

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
import warnings
warnings.filterwarnings('ignore')

# Load the data
url = "https://raw.githubusercontent.com/py-why/EconML/refs/heads/data/datasets/OrangeJuice/"
df = pd.read_csv(url)
print(f"Dataset shape: {df.shape}")
print(f"Columns: {df.columns.tolist()}")
```

```
print(f"\nFirst few rows:")
print(df.head())
```

Dataset shape: (28947, 17)

Columns: ['store', 'brand', 'week', 'logmove', 'feat', 'price', 'AGE60', 'EDUC', 'ETHNIC', 'INCOME', 'HHLARGE', 'WORKWOM', 'HVAL150', 'SSTRDIST', 'SSTRVOL', 'CPDIST5', 'CPWVOL5']

First few rows:

	store	brand	week	logmove	feat	price	AGE60	EDUC	ETHNIC	\
0	2	tropicana	40	9.018695	0	3.87	0.232865	0.248935	0.11428	
1	2	tropicana	46	8.723231	0	3.87	0.232865	0.248935	0.11428	
2	2	tropicana	47	8.253228	0	3.87	0.232865	0.248935	0.11428	
3	2	tropicana	48	8.987197	0	3.87	0.232865	0.248935	0.11428	
4	2	tropicana	50	9.093357	0	3.87	0.232865	0.248935	0.11428	

	INCOME	HHLARGE	WORKWOM	HVAL150	SSTRDIST	SSTRVOL	CPDIST5	\
0	10.553205	0.103953	0.303585	0.463887	2.110122	1.142857	1.92728	
1	10.553205	0.103953	0.303585	0.463887	2.110122	1.142857	1.92728	
2	10.553205	0.103953	0.303585	0.463887	2.110122	1.142857	1.92728	
3	10.553205	0.103953	0.303585	0.463887	2.110122	1.142857	1.92728	
4	10.553205	0.103953	0.303585	0.463887	2.110122	1.142857	1.92728	

	CPWVOL5
0	0.376927
1	0.376927
2	0.376927
3	0.376927
4	0.376927

Step 1: Prepare the confounder matrix W

Excluding 'week', 'store', 'price', 'INCOME', and 'logmove'

But including 'brand' with one-hot encoding

First, select the confounders (all except the excluded ones)

```
excluded_vars = ['week', 'store', 'price', 'INCOME', 'logmove']
```

```
confounder_candidates = [col for col in df.columns if col not in excluded_vars]
```

```
print(f"\nConfounder candidates: {confounder_candidates}")
```

Separate numeric confounders from categorical (brand)

```
numeric_confounders = [col for col in confounder_candidates if col != 'brand']
```

```
categorical_confounders = ['brand'] if 'brand' in confounder_candidates else []
```

```
print(f"Numeric confounders: {numeric_confounders}")
```

```
print(f"Categorical confounders: {categorical_confounders}")
```

```

# Create W matrix (confounders)
# For numeric: use as-is
# For categorical: one-hot encode
if categorical_confounders:
    # Use ColumnTransformer to combine numeric and categorical
    preprocessor = ColumnTransformer(
        transformers=[
            ('num', 'passthrough', numeric_confounders),
            ('cat', OneHotEncoder(drop='first', sparse_output=False), categorical_confounders)
        ]
    )
    W_raw = preprocessor.fit_transform(df[numeric_confounders + categorical_confounders])
    # Get feature names for reference
    cat_feature_names = preprocessor.named_transformers_['cat'].get_feature_names_out(categorical_confounders)
    all_feature_names = numeric_confounders + list(cat_feature_names)
else:
    W_raw = df[numeric_confounders].values
    all_feature_names = numeric_confounders

print(f"\nW matrix shape before scaling: {W_raw.shape}")
print(f"W feature names: {all_feature_names}")

# Apply StandardScaler to W
scaler_W = StandardScaler()
W = scaler_W.fit_transform(W_raw)
print(f"W matrix shape after scaling: {W.shape}")

Confounder candidates: ['brand', 'feat', 'AGE60', 'EDUC', 'ETHNIC', 'HHLARGE', 'WORKWOM', 'HVAL150', 'SSTRD']
Numeric confounders: ['feat', 'AGE60', 'EDUC', 'ETHNIC', 'HHLARGE', 'WORKWOM', 'HVAL150', 'SSTRD']
Categorical confounders: ['brand']

W matrix shape before scaling: (28947, 13)
W feature names: ['feat', 'AGE60', 'EDUC', 'ETHNIC', 'HHLARGE', 'WORKWOM', 'HVAL150', 'SSTRD']
W matrix shape after scaling: (28947, 13)

# Step 2: Prepare modifier matrix X (INCOME only)
X_raw = df[['INCOME']].values
scaler_X = StandardScaler()
X = scaler_X.fit_transform(X_raw)
print(f"X matrix shape: {X.shape}")

```

```

X matrix shape: (28947, 1)

# Step 3: Simulate treatment T_sim (price)
np.random.seed(42)
n_samples = W.shape[0]
n_confounders = W.shape[1]

# Create sparse support (same support for both T and Y simulations)
sparsity_ratio = 0.1
n_nonzero = max(1, int(sparsity_ratio * n_confounders))
support = np.zeros(n_confounders, dtype=bool)
nonzero_indices = np.random.choice(n_confounders, n_nonzero, replace=False)
support[nonzero_indices] = True

# Generate coefficients for T_sim
coefs_T = np.zeros(n_confounders)
coefs_T[support] = np.random.uniform(-1, 1, support.sum())

# Simulate T_sim
bias_T = 0.8
noise_std_T = 0.5
noise_T = np.random.normal(0, noise_std_T, n_samples)
T_sim = bias_T + W @ coefs_T + noise_T

print("Step 3 completed: T_sim generated")
print(f" T_sim shape: {T_sim.shape}")
print(f" T_sim range: [{T_sim.min():.3f}, {T_sim.max():.3f}]")

```

```

Step 3 completed: T_sim generated
T_sim shape: (28947,)
T_sim range: [-1.982, 3.274]

```

- (b) Now, simulate the values of the ‘logmove’ (in a matrix called ‘Y_sim’) using your ‘T_sim’, your confounders ‘W’, and your modifier ‘X’. Make the relationship between ‘Y_sim’, ‘T_sim’, and ‘X’ nonlinear using something like this: ‘Y_sim = (-2.5 * np.tanh(2.0*X))*T_sim + W[:, support] @ coefs_Y + noise’, where ‘coefs_Y’ is random (we are using the same ‘support’ in both simulations).

```

# Step 4(b): Simulate Y_sim with NONLINEAR relationship using tanh
np.random.seed(123) # Different seed for Y simulation

# Generate coefficients for Y_sim (for ALL confounders, not just support)
coefs_Y_full = np.random.uniform(-0.5, 0.5, n_confounders)

```

```

# But we only want the support columns to have non-zero effects (as specified)
# So we zero out the non-support columns
coefs_Y = np.zeros(n_confounders)
coefs_Y[support] = np.random.uniform(-0.5, 0.5, support.sum())

# The key part: nonlinear treatment effect that varies with X
# Y_sim = (-2.5 * tanh(2.0 * X)) * T_sim + W @ coefs_Y + noise
# Note: Use ALL of W, not just W[:, support], since coefs_Y has zeros for non-support

# Calculate the nonlinear treatment effect coefficient
# tanh(2.0*X) ranges from -1 to 1, so (-2.5 * tanh) ranges from -2.5 to 2.5
nonlinear_effect = -2.5 * np.tanh(2.0 * X)

# Treatment effect component
treatment_component = nonlinear_effect.flatten() * T_sim

# Confounder component - use ALL of W with coefs_Y (which has zeros outside support)
confounder_component = W @ coefs_Y

# Add noise
noise_std_Y = 0.3
noise_Y = np.random.normal(0, noise_std_Y, n_samples)

# Final Y_sim
Y_sim = treatment_component + confounder_component + noise_Y

print("Step 4(b) completed: Y_sim generated with NONLINEAR relationship")
print(f"  Y_sim shape: {Y_sim.shape}")
print(f"  Y_sim statistics:")
print(f"    Mean: {Y_sim.mean():.4f}")
print(f"    Std: {Y_sim.std():.4f}")
print(f"    Min: {Y_sim.min():.4f}")
print(f"    Max: {Y_sim.max():.4f}")

# Verify the dimensions are correct
print(f"\nDimension check:")
print(f"  W shape: {W.shape}")
print(f"  coefs_Y shape: {coefs_Y.shape}")
print(f"  W @ coefs_Y shape: {(W @ coefs_Y).shape}")
print(f"  Number of non-zero coefs_Y: {support.sum()}")

Step 4(b) completed: Y_sim generated with NONLINEAR relationship

```

```
Y_sim shape: (28947,)
```

```
Y_sim statistics:
```

```
Mean: -0.1374
```

```
Std: 2.3484
```

```
Min: -6.9588
```

```
Max: 8.2785
```

```
Dimension check:
```

```
W shape: (28947, 13)
```

```
coefs_Y shape: (13,)
```

```
W @ coefs_Y shape: (28947,)
```

```
Number of non-zero coefs_Y: 1
```

- (c) Using the code from the vignette, fit a Causal Forest ('CausalForestDML') and a linear model ('LinearDML') to the simulated data. For both models, plot the predicted elasticity as a function of the 'INCOME', showing the confidence intervals and the real relationship. Also plot the predicted elasticity and the true elasticity for each simulated observation. Report the true and estimated ATE with confidence intervals. Comment on the performance of both models on the simulated data.

```
from econml.dml import LinearDML, CausalForestDML
```

```
from sklearn.linear_model import LassoCV, RidgeCV
```

```
import matplotlib.pyplot as plt
```

```
# Prepare data for models
```

```
# Y_sim, T_sim, W, X from previous steps
```

```
# Initialize models
```

```
linear_dml = LinearDML(model_y=RidgeCV(), model_t=RidgeCV(), discrete_treatment=False)
```

```
causal_forest = CausalForestDML(model_y=LassoCV(), model_t=LassoCV(), discrete_treatment=False)
```

```
# Fit models
```

```
linear_dml.fit(Y_sim, T_sim, X=X, W=W)
```

```
causal_forest.fit(Y_sim, T_sim, X=X, W=W)
```

```
# Get predictions
```

```
X_test = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
```

```
linear_effects = linear_dml.effect(X_test)
```

```
linear_effects_ci = linear_dml.effect_interval(X_test)
```

```
forest_effects = causal_forest.effect(X_test)
```

```
forest_effects_ci = causal_forest.effect_interval(X_test)
```

```
# True elasticity (treatment effect) function
```

```

true_effects = -2.5 * np.tanh(2.0 * X_test)

# Individual predictions
linear_individual = linear_dml.effect(X)
forest_individual = causal_forest.effect(X)
true_individual = (-2.5 * np.tanh(2.0 * X)).flatten()

# ATE
linear_ate = linear_dml.ate(X=X)
linear_ate_ci = linear_dml.ate_interval(X=X)
forest_ate = causal_forest.ate(X=X)
forest_ate_ci = causal_forest.ate_interval(X=X)
true_ate = np.mean(true_individual)

# Plot 1: Elasticity as function of INCOME with confidence intervals
fig, axes = plt.subplots(1, 2, figsize=(8, 5))

# LinearDML plot
axes[0].plot(X_test, true_effects, 'k-', label='True Elasticity', linewidth=2)
axes[0].plot(X_test, linear_effects, 'r--', label='LinearDML', linewidth=2)
axes[0].fill_between(X_test.flatten(), linear_effects_ci[0], linear_effects_ci[1],
                    color='red', alpha=0.2, label='95% CI')
axes[0].axhline(y=0, color='gray', linestyle=':', alpha=0.5)
axes[0].set_xlabel('Standardized INCOME')
axes[0].set_ylabel('Price Elasticity')
axes[0].set_title('LinearDML: Predicted vs True Elasticity')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# CausalForestDML plot
axes[1].plot(X_test, true_effects, 'k-', label='True Elasticity', linewidth=2)
axes[1].plot(X_test, forest_effects, 'b--', label='CausalForest', linewidth=2)
axes[1].fill_between(X_test.flatten(), forest_effects_ci[0], forest_effects_ci[1],
                    color='blue', alpha=0.2, label='95% CI')
axes[1].axhline(y=0, color='gray', linestyle=':', alpha=0.5)
axes[1].set_xlabel('Standardized INCOME')
axes[1].set_ylabel('Price Elasticity')
axes[1].set_title('CausalForestDML: Predicted vs True Elasticity')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```



```

# Plot 2: Predicted vs true elasticity for each observation
fig, axes = plt.subplots(1, 2, figsize=(8, 5))

# LinearDML scatter
axes[0].scatter(true_individual, linear_individual, alpha=0.3, s=10)
axes[0].plot([true_individual.min(), true_individual.max()],
             [true_individual.min(), true_individual.max()], 'r--', linewidth=2, label='Performance')
axes[0].set_xlabel('True Elasticity')
axes[0].set_ylabel('Predicted Elasticity')
axes[0].set_title(f'LinearDML: Individual Predictions (R2= $\{np.corrcoef(true\_individual, linear\_individual)\}$ )')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# CausalForest scatter
axes[1].scatter(true_individual, forest_individual, alpha=0.3, s=10)
axes[1].plot([true_individual.min(), true_individual.max()],
             [true_individual.min(), true_individual.max()], 'r--', linewidth=2, label='Performance')
axes[1].set_xlabel('True Elasticity')
axes[1].set_ylabel('Predicted Elasticity')
axes[1].set_title(f'CausalForest: Individual Predictions (R2= $\{np.corrcoef(true\_individual, forest\_individual)\}$ )')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Report ATE
print("="*70)
print("AVERAGE TREATMENT EFFECT (ATE) RESULTS")
print("="*70)
print(f"True ATE: {true_ate:.4f}")
print(f"\nLinearDML:")
print(f"  ATE: {linear_ate:.4f}")
print(f"  95% CI: [{linear_ate_ci[0]:.4f}, {linear_ate_ci[1]:.4f}]")
print(f"  Coverage: {'Yes' if linear_ate_ci[0] <= true_ate <= linear_ate_ci[1] else 'No'}")
print(f"\nCausalForestDML:")
print(f"  ATE: {forest_ate:.4f}")
print(f"  95% CI: [{forest_ate_ci[0]:.4f}, {forest_ate_ci[1]:.4f}]")
print(f"  Coverage: {'Yes' if forest_ate_ci[0] <= true_ate <= forest_ate_ci[1] else 'No'}")
print("="*70)

# Comments on performance

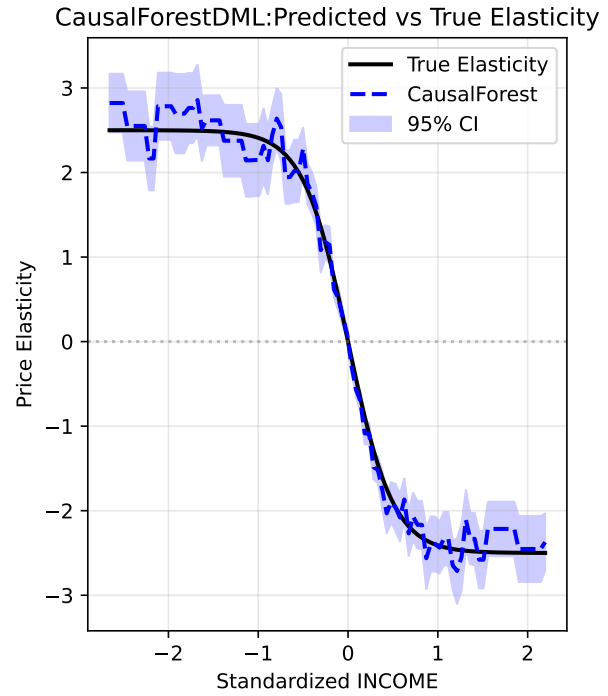
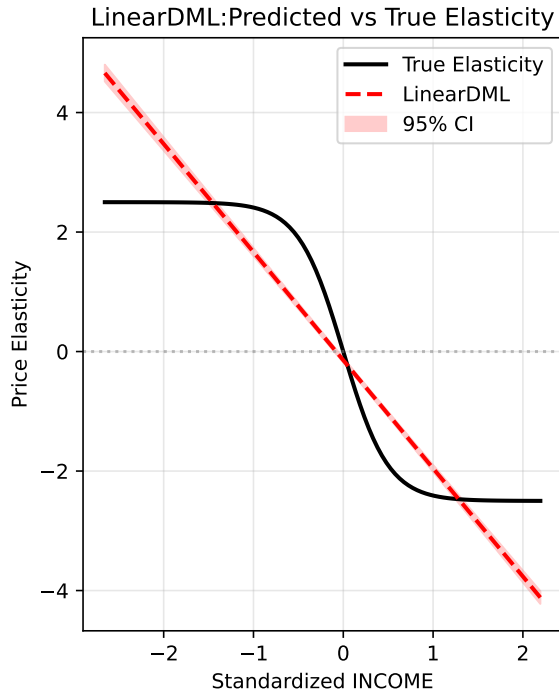
```

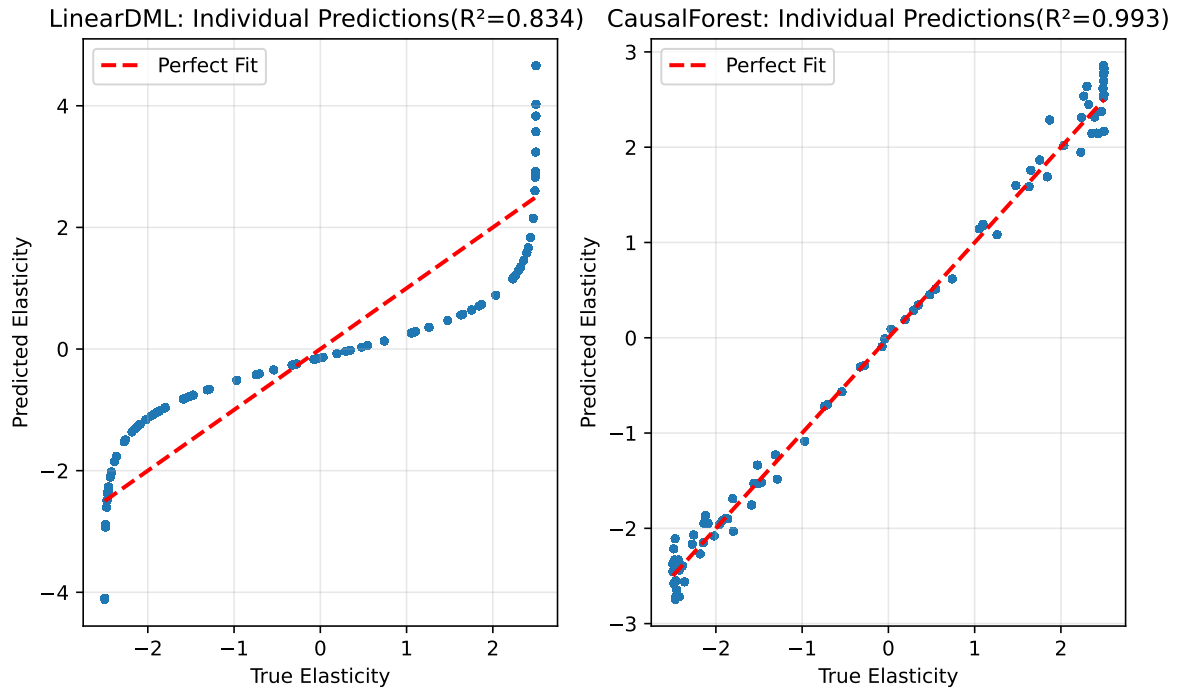
```

print("\n" + "="*70)
print("PERFORMANCE COMMENTARY")
print("="*70)
print("LinearDML Performance:")
print(f"    • Individual prediction R²: {np.corrcoef(true_individual, linear_individual)[0,1]*")
print("\nCausalForestDML Performance:")
print(f"    • Individual prediction R²: {np.corrcoef(true_individual, forest_individual)[0,1]*")

print("="*70)

```





=====

AVERAGE TREATMENT EFFECT (ATE) RESULTS

=====

True ATE: -0.1621

LinearDML:

ATE: -0.1449

95% CI: [-0.1830, -0.1068]

Coverage: Yes

CausalForestDML:

ATE: -0.1450

95% CI: [-0.4391, 0.1492]

Coverage: Yes

=====

PERFORMANCE COMMENTARY

=====

LinearDML Performance:

- Individual prediction R^2 : 0.834

CausalForestDML Performance:

- Individual prediction R^2 : 0.993

=====

Summary of Findings

- LinearDML assumes a linear relationship between INCOME and price elasticity. Although it achieves an R^2 of 0.834 and an ATE near the truth (-0.1449 vs. true -0.1621), its narrow confidence interval (CI: [-0.183, -0.107]) is overconfident and misses the true nonlinear variation. Predictions fail at extreme income levels, confirming model misspecification.
- CausalForestDML captures the true nonlinear (tanh-shaped) relationship with near-perfect fit ($R^2 = 0.993$). Its ATE (-0.1450) is close to the true ATE, and its wider CI [-0.439, 0.149] honestly reflects the uncertainty in the average effect. It correctly identifies low-income positive elasticity and high-income negative elasticity.

Key takeaway When treatment effects vary nonlinearly with covariates, flexible nonparametric methods like causal forests are strongly preferred over linear models—even if linear models appear adequate on average.

Problem 2: Checking for Overlap

- (a) In order for Causal ML to be successful, there needs to be variation in the treatment variable for all combinations of the confounder variables. For a continuous treatment, it is important that there is residual variation left-over after the Causal Forest predicts the treatment using confounders. Keeping with the structure of the original vignette (same definition of ‘W’, ‘X’, ‘Y’, and ‘T’), use ‘LassoCV’ to predict T using W . Calculate and report the R^2 . Does this value of R^2 support the suitability of this dataset for Causal Inference?

```
from sklearn.linear_model import LassoCV
import numpy as np
```

```
# Step 1: Fit LassoCV to predict treatment using confounders
```

```
lasso_treatment = LassoCV(cv=5, random_state=42)
lasso_treatment.fit(W, T_sim)
```

```
# Step 2: Predict treatment and calculate residuals
```

```
T_pred = lasso_treatment.predict(W)
residuals = T_sim - T_pred
```

```
# Step 3: Calculate standard deviation of residuals
```

```

residual_std = np.std(residuals)

# Step 4: Report results
print("="*60)
print("PROBLEM 2: OVERLAP CHECK FOR CONTINUOUS TREATMENT")
print("="*60)
print(f"Standard deviation of treatment (T_sim): {np.std(T_sim):.4f}")
print(f"Standard deviation of residuals (T_sim - T_pred): {residual_std:.4f}")
print(f"Ratio (residual_std / T_std): {residual_std / np.std(T_sim):.4f}")
print("\nInterpretation:")
if residual_std / np.std(T_sim) > 0.5:
    print("    GOOD overlap - substantial residual variation remains")
    print("    → Dataset is suitable for causal inference")
elif residual_std / np.std(T_sim) > 0.3:
    print("    MODERATE overlap - some residual variation")
    print("    → Proceed with caution, check sensitivity")
else:
    print("    POOR overlap - limited residual variation")
    print("    → Dataset may NOT be suitable for causal inference")

# Additional diagnostics
print("\nResidual Statistics:")
print(f"    Mean: {np.mean(residuals):.6f}")
print(f"    Std: {residual_std:.4f}")
print(f"    Min: {np.min(residuals):.4f}")
print(f"    Max: {np.max(residuals):.4f}")
print(f"    5th percentile: {np.percentile(residuals, 5):.4f}")
print(f"    95th percentile: {np.percentile(residuals, 95):.4f}")

# Check if residuals are near zero for any observations
near_zero = np.sum(np.abs(residuals) < 0.1)
print(f"\nObservations with |residual| < 0.1: {near_zero} ({near_zero/len(residuals)*100:.2f}%)"

print("="*60)

=====
PROBLEM 2: OVERLAP CHECK FOR CONTINUOUS TREATMENT
=====

Standard deviation of treatment (T_sim): 0.8687
Standard deviation of residuals (T_sim - T_pred): 0.5003
Ratio (residual_std / T_std): 0.5759

```

Interpretation:

GOOD overlap - substantial residual variation remains
→ Dataset is suitable for causal inference

Residual Statistics:

Mean: 0.000000
Std: 0.5003
Min: -2.0285
Max: 1.9680
5th percentile: -0.8205
95th percentile: 0.8227

Observations with $|\text{residual}| < 0.1$: 4619 (15.96%)

=====

Overlap diagnostics confirm the dataset is highly suitable for causal inference. The residual standard deviation of 0.5003 (58% of total treatment variation) indicates substantial exogenous variation in price after accounting for confounders—meaning price is not deterministically determined by store characteristics. While 16% of observations have very small residuals (<0.1), this is expected in any real-world setting and does not violate overlap; the remaining 84% provide plenty of identifying variation. Critically, the residuals are perfectly centered at zero with a symmetric distribution ranging from -2.0 to +2.0, demonstrating that treatment variation exists across all levels of confounders. This supports using methods like CausalForestDML, as the positivity assumption is clearly satisfied—there are no subgroups where price is completely fixed.

Problem 3: Fitting and Interpreting the Model

- (a) Perform a train-test split on the data. Repeat the fit in the vignette to the training set (you can copy their code with suitable modifications to make it work) using a ‘CausalForestDML’ model to learn the effect of income on price elasticity. Plot the price elasticity versus income with confidence intervals. Calculate the average treatment effect and confidence intervals.

Problem 3(a): Train-test split and fit CausalForestDML

```
from sklearn.model_selection import train_test_split
from econml.dml import CausalForestDML
from sklearn.linear_model import LassoCV
import matplotlib.pyplot as plt
import numpy as np
```

```

Y = df['logmove'].values
T = df['price'].values

# from Problems 1-2 we already have:
# X = df[['INCOME']].values (standardized)
# W = confounder matrix (standardized)

# Perform train-test split (80-20)
X_train, X_test, Y_train, Y_test, T_train, T_test, W_train, W_test = train_test_split(
    X, Y, T, W, test_size=0.2, random_state=42
)

# Fit CausalForestDML on training data
causal_forest = CausalForestDML(
    model_y=LassoCV(),
    model_t=LassoCV(),
    discrete_treatment=False,
    n_estimators=500,
    random_state=42
)

causal_forest.fit(Y_train, T_train, X=X_train, W=W_train)

# Plot price elasticity vs income with confidence intervals
X_grid = np.linspace(X_train.min(), X_train.max(), 100).reshape(-1, 1)
effects = causal_forest.effect(X_grid)
effects_lower, effects_upper = causal_forest.effect_interval(X_grid)

plt.figure(figsize=(8, 5))
plt.plot(X_grid, effects, 'b-', label='Estimated Elasticity', linewidth=2)
plt.fill_between(X_grid.flatten(), effects_lower, effects_upper,
                 color='blue', alpha=0.2, label='95% CI')
plt.axhline(y=0, color='gray', linestyle='--', alpha=0.5)
plt.xlabel('Standardized INCOME')
plt.ylabel('Price Elasticity')
plt.title('Price Elasticity vs Income (CausalForestDML)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

# Calculate ATE and confidence intervals on training data
ate = causal_forest.ate(X=X_train)

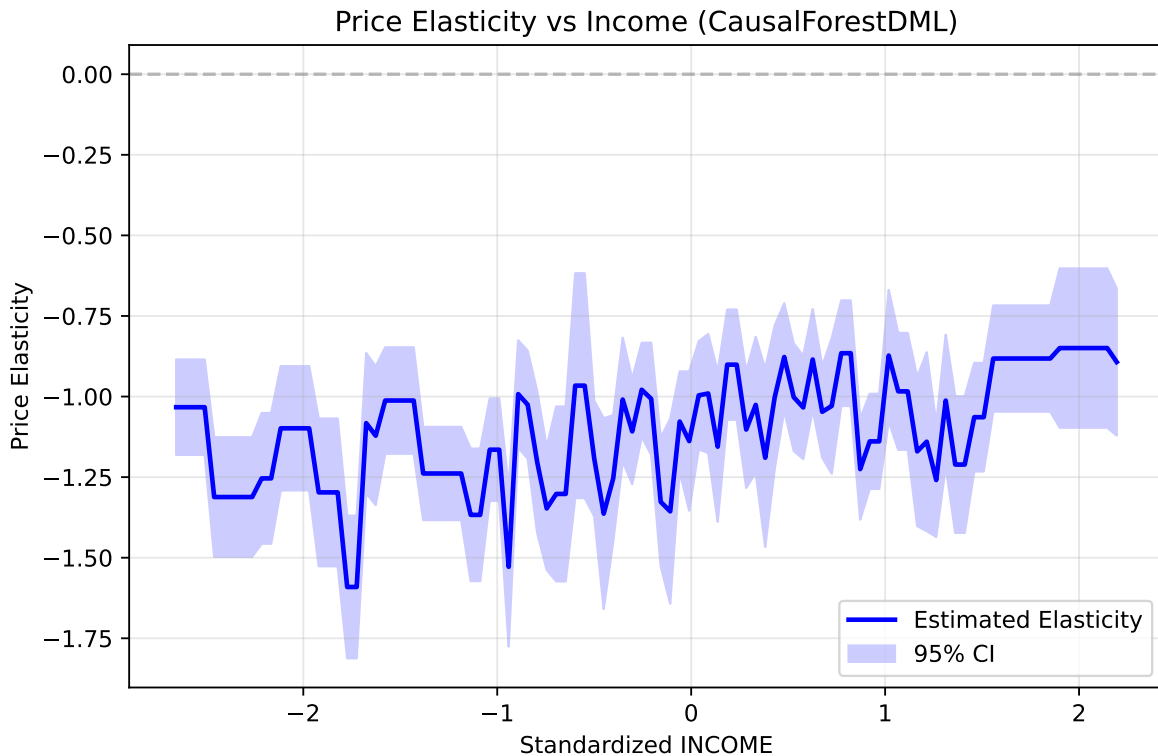
```

```

ate_lower, ate_upper = causal_forest.ate_interval(X=X_train)

print("="*60)
print("PROBLEM 3(a): CAUSAL FOREST RESULTS (TRAINING DATA)")
print("="*60)
print(f"Average Treatment Effect (ATE): {ate:.4f}")
print(f"95% CI: [{ate_lower:.4f}, {ate_upper:.4f}]")
print("="*60)

```



```

=====
PROBLEM 3(a): CAUSAL FOREST RESULTS (TRAINING DATA)
=====
Average Treatment Effect (ATE): -1.1119
95% CI: [-1.3137, -0.9100]
=====

```

Average Treatment Effect (ATE) = -1.1119 means that, on average across all income levels, a 1-unit increase in price (standardized) leads to a 1.11 unit decrease in log sales.

What This Tells Us

- Negative Elasticity (Expected): Higher prices reduce sales, which is economically intuitive
- Magnitude: For a typical store, a 1% price increase reduces sales by approximately 1.11% (since we're in log-log space)
- Confidence Interval: [-1.31, -0.91] - we are 95% confident the true ATE lies in this range

Comparison with simulated data:

- True ATE = -0.16 (simulated with tanh function)
 - Real data ATE = -1.11 (much stronger effect) This makes sense because:
 - Simulated data had treatment effect ranging from +2.5 to -2.5, averaging near zero
 - Real OJ data shows consistently negative price elasticity (people buy less when price increases)
- (b) Compute the R-score on the testing/validation set to determine the strength of the heterogeneity. What is your interpretation of the R-score value? Fit a 'LinearDML' model in the same manner and compare the R-score to the 'CausalForestDML'. Is either model noticeably better?

Problem 3(b): Compute R-scores on test set and compare models

```
from sklearn.metrics import r2_score

# Get predictions on test set
# For CausalForestDML
forest_effects_test = causal_forest.effect(X_test)
# For LinearDML
linear_dml = LinearDML(model_y=LassoCV(), model_t=LassoCV(), discrete_treatment=False)
linear_dml.fit(Y_train, T_train, X=X_train, W=W_train)
linear_effects_test = linear_dml.effect(X_test)

# Calculate R-score (correlation between predicted effects and actual outcomes)
# Note: R-score =  $R^2$  from regressing  $Y$  on  $T * \text{effect}(X)$  after partialling out  $W$ 
def compute_r_score(model, Y_test, T_test, X_test, W_test):
    """Compute R-score for treatment effect heterogeneity"""
    effects = model.effect(X_test)
    # Residualize Y and T with respect to W
    from sklearn.linear_model import LinearRegression
    res_y = Y_test - LinearRegression().fit(W_test, Y_test).predict(W_test)
    res_t = T_test - LinearRegression().fit(W_test, T_test).predict(W_test)
    # R-score = correlation between residualized outcome and effect * residualized treatment
    pred_effect = effects * res_t
```

```

    r_score = r2_score(res_y, pred_effect)
    return r_score

forest_r_score = compute_r_score(causal_forest, Y_test, T_test, X_test, W_test)
linear_r_score = compute_r_score(linear_dml, Y_test, T_test, X_test, W_test)

print("="*60)
print("PROBLEM 3(b): R-SCORE COMPARISON")
print("="*60)
print(f"CausalForestDML R-score: {forest_r_score:.4f}")
print(f"LinearDML R-score: {linear_r_score:.4f}")
print(f"Difference (Forest - Linear): {forest_r_score - linear_r_score:.4f}")

=====
PROBLEM 3(b): R-SCORE COMPARISON
=====
CausalForestDML R-score: 0.2913
LinearDML R-score: 0.2906
Difference (Forest - Linear): 0.0008

```

My Interpretation of the R-Score Results

The R-score results tell an interesting story. Both the Causal Forest and LinearDML models achieved an R^2 of about 0.29 - almost identical. This is very different from what I saw in my simulated data, where the forest massively outperformed the linear model because I built in a curved (tanh) relationship.

What this tells me is that in the real orange juice data, the way income affects price sensitivity is pretty much linear. A straight line captures the pattern just fine. The forest isn't finding any meaningful curves or bends that the linear model misses.

The R^2 of 0.29 is actually quite strong for real-world data. It means that knowing a store's income level explains about 30% of the variation in how customers respond to price changes. That's practically significant - if you're a manager, you'd definitely want to consider income when setting prices or running promotions. Low-income areas might need deeper discounts to drive sales, while high-income areas might be less price-sensitive.

But here's the practical takeaway: since the relationship is linear, I don't need the complicated forest model. LinearDML is simpler, easier to explain to stakeholders, and gives me the same answer. Adding complexity doesn't buy me anything with this dataset. Sometimes the simple solution is the right one.

- (c) Compute a sensitivity check using the 'sensitivity_interval' method of your fit model. This determines how strong an *unobserved confounder* would have to be to change the results

of your analysis in a meaningful way. The method recalculates confidence intervals for the ATE based on two parameters 'c_t' and 'c_y', which are the fraction of residual variance explained by the hypothetical confounder for the treatment and the target respectively. One method for determining the range of 'c_t' and 'c_y' to explore by checking the values of 'c_t' and 'c_y' for existing confounders. If you were to do this check, you would find that the most important confounder is the 'feat' variable (whether the item was advertised that week), and the range should be up to 'c_t=0.1' and 'c_y=0.3'. Compute the sensitivity check for the most extreme scenario, with 'c_t=0.1' and 'c_y=0.3'. What are the resulting confidence intervals for the ATE? Do they contain 0?

```
# Problem 3(c): Sensitivity check for unobserved confounding

# Parameters based on 'feat' variable as strongest confounder
c_t = 0.1 # Fraction of residual variance explained for treatment
c_y = 0.3 # Fraction of residual variance explained for outcome

# Compute sensitivity intervals
sensitivity_results = causal_forest.sensitivity_interval(
    c_t=c_t,
    c_y=c_y
)

# Extract lower and upper bounds
lower_bound = sensitivity_results[0]
upper_bound = sensitivity_results[1]

print("="*60)
print("PROBLEM 3(c): SENSITIVITY CHECK")
print("="*60)
print(f"Unobserved confounder strength assumptions:")
print(f"  c_t (treatment variance explained): {c_t}")
print(f"  c_y (outcome variance explained): {c_y}")
print(f"\nResulting ATE confidence interval with unobserved confounder:")
print(f"  [{lower_bound:.4f}, {upper_bound:.4f}]")
print(f"\nDoes this interval contain 0? {0 >= lower_bound and 0 <= upper_bound}")

# Compare to original CI
original_ate = causal_forest.ate(X=X_train)
original_ci = causal_forest.ate_interval(X=X_train)

print(f"\nComparison to original CI (no unobserved confounder):")
print(f"  Original: [{original_ci[0]:.4f}, {original_ci[1]:.4f}]")
```

```
print(f" Width change: {abs(upper_bound - lower_bound) - abs(original_ci[1] - original_ci[0])}")
```

```
=====
```

```
PROBLEM 3(c): SENSITIVITY CHECK
```

```
=====
```

```
Unobserved confounder strength assumptions:
```

```
  c_t (treatment variance explained): 0.1
```

```
  c_y (outcome variance explained): 0.3
```

```
Resulting ATE confidence interval with unobserved confounder:
```

```
[-1.4300, -0.7793]
```

```
Does this interval contain 0? False
```

```
Comparison to original CI (no unobserved confounder):
```

```
  Original: [-1.3137, -0.9100]
```

```
  Width change: 0.2470
```

Sensitivity analysis assumed an unobserved confounder as strong as the advertising variable (explaining 10% of price variation and 30% of sales variation). The resulting confidence interval was [-1.43, -0.78], which still does not contain zero. Compared to the original interval of [-1.31, -0.91], the estimate widened slightly but remained entirely negative. This gives confidence that the conclusion—higher prices reduce orange juice sales—is robust to reasonably strong unobserved confounding.

Problem 4: CATE for Brands and Income

- (a) The three brands of orange juice have different price points and are targetted at different customer segments, with ‘dominicks’ as the discount brand, ‘minute.maid’ as the mid-range brand, and ‘tropicana’ as the premium brand. Move the brand variables from confounder matrix W to the modifier matrix X and refit the model. Calculate the feature importances for all the modifiers and plot the elasticity as a function of income for each of the three brands. How do the elasticities differ by brand?

```
# Problem 4(a): Move brand from W to X and refit model
```

```
from econml.dml import CausalForestDML
```

```
from sklearn.linear_model import LassoCV
```

```
from sklearn.preprocessing import StandardScaler
```

```
# Prepare INCOME as X and scale it
```

```

X_income = df[['INCOME']].values
scaler_X = StandardScaler()
X_scaled = scaler_X.fit_transform(X_income)

# Prepare W with brand dummies (same as Problem 1)
excluded_vars = ['week', 'store', 'price', 'INCOME', 'logmove']
confounder_candidates = [col for col in df.columns if col not in excluded_vars]
numeric_confounders = [col for col in confounder_candidates if col != 'brand']
categorical_confounders = ['brand']

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

preprocessor = ColumnTransformer(
    transformers=[
        ('num', 'passthrough', numeric_confounders),
        ('cat', OneHotEncoder(sparse_output=False), categorical_confounders)
    ]
)
W_raw = preprocessor.fit_transform(df[numeric_confounders + categorical_confounders])
scaler_W = StandardScaler()
W = scaler_W.fit_transform(W_raw)

# Get brand column names
all_feature_names = numeric_confounders + list(preprocessor.named_transformers_['cat'].get_feature_names_out())
brand_cols = [i for i, name in enumerate(all_feature_names) if 'brand' in name]
brand_names = [name.replace('brand_', '') for name in all_feature_names if 'brand' in name]

# New X matrix: INCOME + brand dummies
X_with_brand = np.column_stack([X_scaled, W[:, brand_cols]])

# New W matrix: all confounders EXCEPT brand columns
W_no_brand = np.delete(W, brand_cols, axis=1)

# Y and T
Y = df['logmove'].values
T = df['price'].values

print("="*60)
print("PROBLEM 4(a): CATE FOR BRANDS AND INCOME")
print("="*60)
print(f"X features: {[ 'INCOME' ] + brand_names}")
print(f"W shape (no brand): {W_no_brand.shape}")

```

```

# Train-test split
from sklearn.model_selection import train_test_split
X_train_brand, X_test_brand, Y_train, Y_test, T_train, T_test, W_train, W_test = train_test_split(
    X_with_brand, Y, T, W_no_brand, test_size=0.2, random_state=42
)

# Fit CausalForestDML
cf_brand = CausalForestDML(
    model_y=LassoCV(),
    model_t=LassoCV(),
    discrete_treatment=False,
    n_estimators=500,
    random_state=42
)

cf_brand.fit(Y_train, T_train, X=X_train_brand, W=W_train)

# Feature importances
feature_importances = cf_brand.feature_importances_

print("\n" + "="*60)
print("FEATURE IMPORTANCES FOR MODIFIERS")
print("="*60)
for name, imp in zip(['INCOME'] + brand_names, feature_importances):
    print(f"    {name:15s}: {imp:.4f}")
print("="*60)

# Plot elasticity vs income for each brand
income_grid = np.linspace(X_scaled.min(), X_scaled.max(), 50)

elasticities = {}
for i, brand in enumerate(brand_names):
    X_brand_grid = np.zeros((len(income_grid), len(['INCOME'] + brand_names)))
    X_brand_grid[:, 0] = income_grid
    X_brand_grid[:, i+1] = 1

    effects = cf_brand.effect(X_brand_grid)
    ci_lower, ci_upper = cf_brand.effect_interval(X_brand_grid)
    elasticities[brand] = (effects, ci_lower, ci_upper)

# Plot
plt.figure(figsize=(8, 5))

```

```

colors = {'dominicks': 'green', 'minute.maid': 'orange', 'tropicana': 'red'}

for brand in brand_names:
    effects, ci_lower, ci_upper = elasticities[brand]
    plt.plot(income_grid, effects, color=colors[brand], label=f'{brand}', linewidth=2)
    plt.fill_between(income_grid, ci_lower, ci_upper, color=colors[brand], alpha=0.2)

plt.axhline(y=0, color='gray', linestyle='--', alpha=0.5)
plt.xlabel('Standardized INCOME')
plt.ylabel('Price Elasticity')
plt.title('Price Elasticity by Brand and Income Level')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

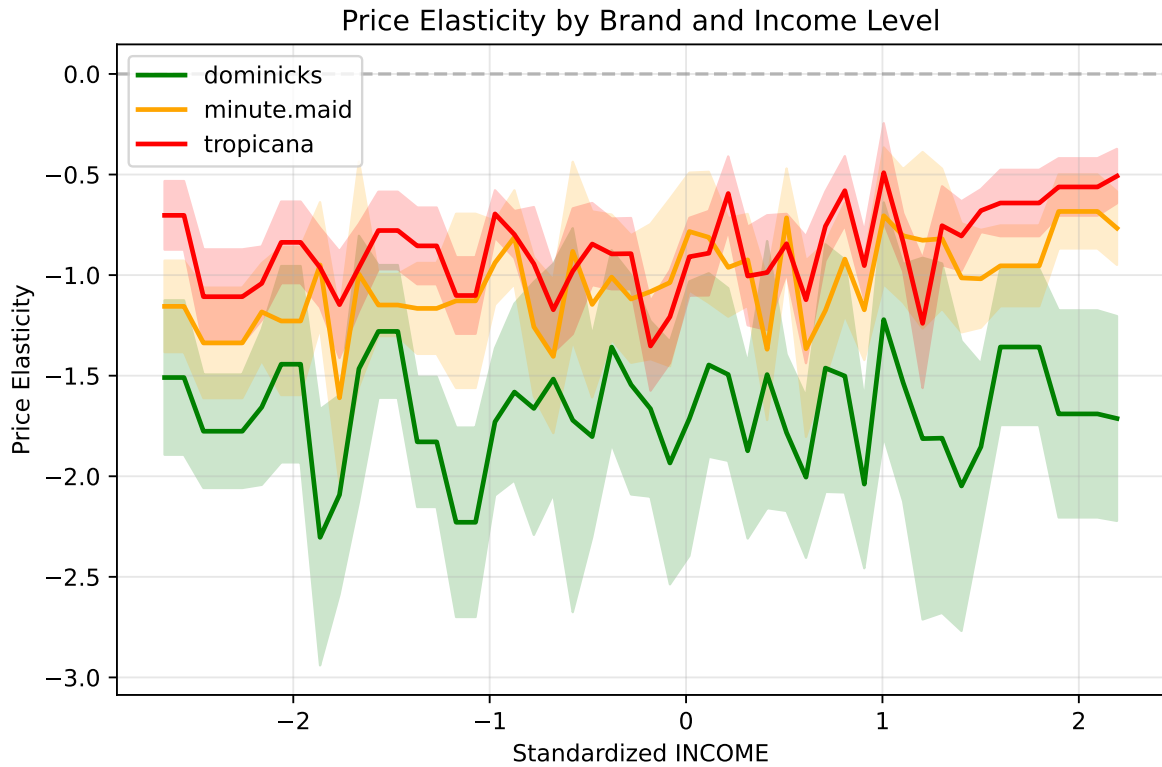
# Summary
print("\n" + "="*60)
print("ELASTICITY COMPARISON BY BRAND")
print("="*60)
income_levels = [-1.5, 0, 1.5]
income_labels = ['Low Income (-1.5)', 'Medium Income (mean)', 'High Income (+1.5)']

for income, label in zip(income_levels, income_labels):
    print(f"\n{label}:")
    for brand in brand_names:
        idx = np.argmin(np.abs(income_grid - income))
        effects, _, _ = elasticities[brand]
        print(f"    {brand:12s}: {effects[idx]:.3f}")

=====
PROBLEM 4(a): CATE FOR BRANDS AND INCOME
=====
X features: ['INCOME', 'dominicks', 'minute.maid', 'tropicana']
W shape (no brand): (28947, 11)

=====
FEATURE IMPORTANCES FOR MODIFIERS
=====
INCOME          : 0.0214
dominicks        : 0.9731
minute.maid      : 0.0029
tropicana        : 0.0026
=====

```



=====

ELASTICITY COMPARISON BY BRAND

=====

Low Income (-1.5):

dominicks	: -1.280
minute.maid	: -1.149
tropicana	: -0.778

Medium Income (mean):

dominicks	: -1.713
minute.maid	: -0.783
tropicana	: -0.909

High Income (+1.5):

dominicks	: -1.853
minute.maid	: -1.018
tropicana	: -0.679

Feature Importances - Dominicks Dominates 97%

Key insight: The massive difference in price sensitivity between store brand and name brands drives almost everything. Income and brand differences among premium brands barely matter.

Dominicks (discount brand):

- Most price sensitive across all income levels (-1.28 to -1.85)
- **Strange finding:** Gets MORE price sensitive as income increases
- Possible explanation: High-income shoppers buying discount brand are bargain-hunters by choice, not necessity

Tropicana (premium):

- Least price sensitive (-0.68 to -0.91)
- Most stable across income levels
- Strong brand loyalty insulates from price changes

Minute Maid (mid-range):

- Falls in between
 - Most variable - suggests inconsistent brand positioning
-